Table 1: Syntax with some commonly used examples

Syntax	Examples
Sizeof (type-name)	1. sizeof (char) 2. sizeof (int) 3. sizeof (float) 4. sizeof (double) 5. sizeof (12.5) 6. sizeof ('A') etc...

Table 2: Syntax with some commonly used examples

Syntax	Examples
Sizeof (expression) *OR* Sizeof expression	1. sizeof (i++) 2. sizeof (a+b) etc...

```
10        return 0;
11 }
```

Note 1: The return value of the *sizeof()* operator is implementation-defined, and its type (an unsigned integer type) is *size_t*, defined in <*stddef.h*>.

Note 2: C99 has included *%zu* as a type specifier for size_t. But for older compilers, %z will fail. Hence, use *%lu* or *%llu* along with a typecasting to achieve portability of the code across various platforms.

Case 2: When the operand is an expression

When *sizeof()* is used with expression as an operand, the operand can be enclosed with or without parenthesis. The syntax of how the *sizeof()* operator is used preceding the expression is shown in Table 2 along with some examples.

Illustration: The sample demo code is shown in the Code 2 snippet, and the output of the code when I run it in my system is shown in Figure 2.

Code 2

```
 1 #include <stdio.h>
 2 #include <stddef.h>
 3
 4 int main()
 5 {
 6         int a = 10;
 7
 8         double d = 12.34;
 9
10         printf("Sizeof( a + d ): %lu\n", (long unsigned)
sizeof(a + d));
11
12         return 0;
```

```
satya@Elegance: ~/Desktop/OSFY:Articles
[satya]
./a.out
Sizeof(char)   : 1

Sizeof(int)    : 4

Sizeof(float) : 4

Sizeof(double): 8
```

Figure 1: Output of the demo program shown in Code 1

```
satya@Elegance: ~/Desktop/OSFY:Articles
[satya]
./a.out
Sizeof( a + d ) : 8
```

Figure 2: Output of the demo program shown in Code 2

```
satya@Elegance: ~/Desktop/OSFY:Articles
[satya]
./a.out
Size of i : 4
Value of i : 10
[satya]
```

Figure 3: Output of the demo program shown in Code 3

```
13 }
```

From the above demo code, it is very clear that when the *sizeof()* operator is applied to an expression, it yields a result that is the same as if it had been applied to the type-name of the resultant of the expression. Since, at compile time the compiler analyses the expression to determine its type, but it will never evaluate the expression which takes place at runtime.

In the example shown in Code 2, 'a' is of *int* type and'd' is of double type. When type conversion is applied, as usual, the lower rank data type is promoted to a higher rank data type and the resultant data type is nothing but a double in our case; hence, *sizeof(a + d)* yields *sizeof(double)* which is 8 bytes as shown in Figure 2. In general, if the operand contains the operators that perform type conversions, the compiler considers these conversions in determining the type of the expression.

Behaviour

The *sizeof()* operator behaves differently in comparison with other operators. In this article, let me point out the uniqueness of this operator by taking two real-time programming examples. The first is about compile-time behaviour and the second one is about runtime behaviour.

Case 1: Compile-time behaviour

To start with, let us consider the simple code as shown in the Code 3 snippet.